

# MYSQL STORED PROCEDURES

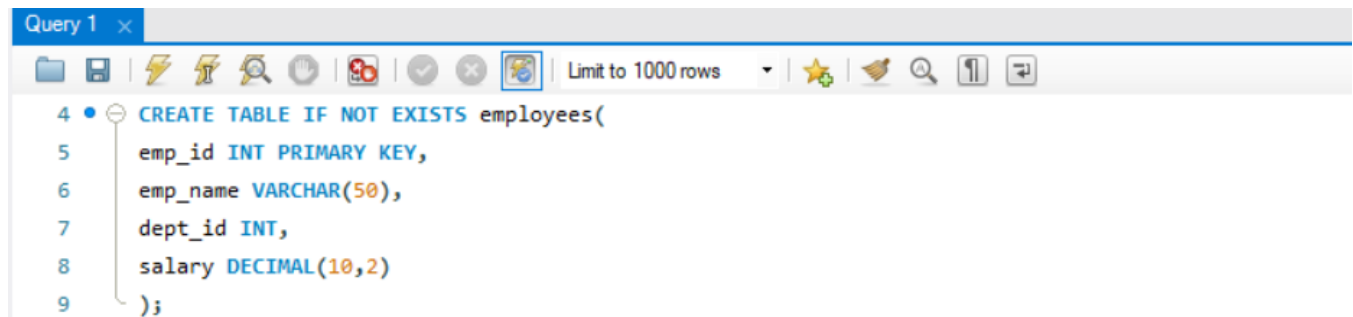
*Examples and Practice Exercises for MySQL Workbench*

## 1. Setup: Employees Table

```
CREATE DATABASE IF NOT EXISTS employeedb1;  
USE employeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

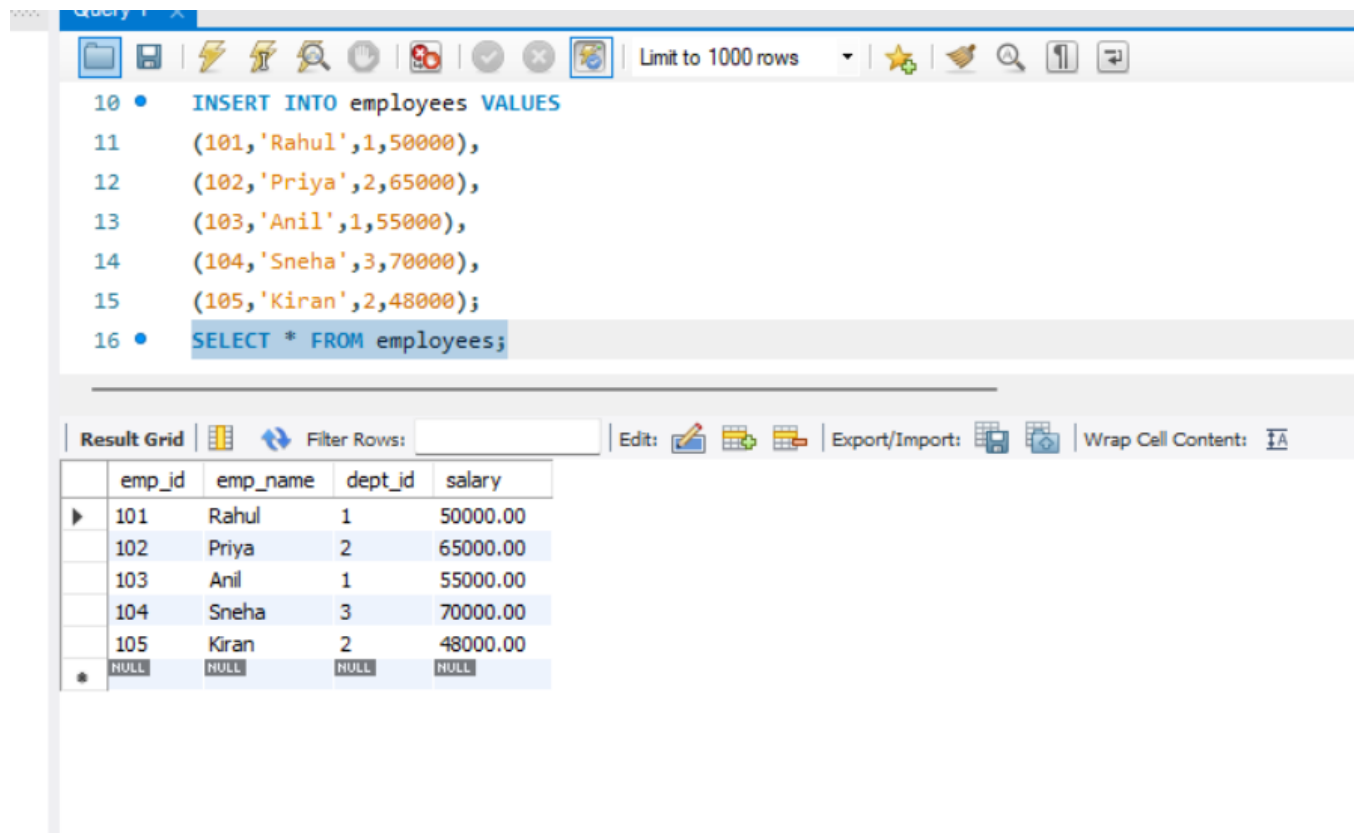


## 2. Verify Data

Question: Display all employee records.

```
SELECT * FROM employees;
```

## OUTPUT:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query window contains the following SQL code:

```
10 • INSERT INTO employees VALUES
11     (101, 'Rahul', 1, 50000),
12     (102, 'Priya', 2, 65000),
13     (103, 'Anil', 1, 55000),
14     (104, 'Sneha', 3, 70000),
15     (105, 'Kiran', 2, 48000);
16 • SELECT * FROM employees;
```

Below the query window is the 'Result Grid' section. It includes a 'Filter Rows' input field and buttons for 'Edit', 'Export/Import', and 'Wrap Cell Content'. The result grid displays the following data:

|   | emp_id | emp_name | dept_id | salary   |
|---|--------|----------|---------|----------|
| ▶ | 101    | Rahul    | 1       | 50000.00 |
|   | 102    | Priya    | 2       | 65000.00 |
|   | 103    | Anil     | 1       | 55000.00 |
|   | 104    | Sneha    | 3       | 70000.00 |
|   | 105    | Kiran    | 2       | 48000.00 |
| * | NULL   | NULL     | NULL    | NULL     |

### Procedure 1 – Display All Employees

**Question:** Create a procedure that displays all employees.

```
DROP PROCEDURE IF EXISTS GetEmployees;
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM employees;
END $$
```

```
DELIMITER ;
```

```
CALL GetEmployees();
```

## OUTPUT:

The screenshot displays a SQL IDE interface. The top pane shows the following SQL code:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE GetEmployees()
4 BEGIN
5     SELECT * FROM employees;
6 END $$
7 DELIMITER ;
```

The bottom pane shows the 'Result Grid' with the following data:

| emp_id | emp_name | dept_id | salary   |
|--------|----------|---------|----------|
| 101    | Rahul    | 1       | 50000.00 |
| 102    | Priya    | 2       | 65000.00 |
| 103    | Anil     | 1       | 55000.00 |
| 104    | Sneha    | 3       | 70000.00 |
| 105    | Kiran    | 2       | 48000.00 |

Below the result grid, the 'Output' pane shows the 'Action Output' with the following log:

| # | Time     | Action                                                                                                        | Message                                                |
|---|----------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 4 | 11:28:00 | USE employeeedb1                                                                                              | 0 row(s) affected                                      |
| 5 | 11:31:45 | CREATE TABLE IF NOT EXISTS employees(emp_id INT PRIMARY KEY, emp_name VARCHAR(50), dept_id...                 | 0 row(s) affected                                      |
| 6 | 11:45:15 | INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sne... | 5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0 |
| 7 | 11:46:37 | SELECT * FROM employees LIMIT 0, 1000                                                                         | 5 row(s) returned                                      |
| 8 | 12:01:42 | CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; END                                            | 0 row(s) affected                                      |
| 9 | 12:02:23 | CALL GetEmployees()                                                                                           | 5 row(s) returned                                      |

### Procedure 2 – IN Parameter

**Question:** Create a procedure that accepts an employee ID and displays that employee's details.

```
DROP PROCEDURE IF EXISTS GetEmployeeById;
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployeeById(IN eid INT)
BEGIN
    SELECT * FROM employees
    WHERE emp_id = eid;
END $$
```

```
DELIMITER ;
```

```
CALL GetEmployeeById(102);
```

Query 1 x

Limit to 1000 rows

```

3 • CREATE PROCEDURE GetEmployeeById(IN eid INT)
4 BEGIN
5     SELECT * FROM employees
6     WHERE emp_id = eid;
7 END $$
8 DELIMITER ;
9 • CALL GetEmployeeById(102);

```

Result Grid

| emp_id | emp_name | dept_id | salary   |
|--------|----------|---------|----------|
| 102    | Priya    | 2       | 65000.00 |

Result 4 x

Output

Action Output

| #  | Time     | Action                                                                                     | Message                                                    |
|----|----------|--------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 7  | 11:46:37 | SELECT * FROM employees LIMIT 0, 1000                                                      | 5 row(s) returned                                          |
| 8  | 12:01:42 | CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; END                         | 0 row(s) affected                                          |
| 9  | 12:02:23 | CALL GetEmployees()                                                                        | 5 row(s) returned                                          |
| 10 | 12:11:12 | CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM employees WHERE emp id... | Error Code: 1064. You have an error in your SQL syntax; ch |
| 11 | 12:11:48 | CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM employees WHERE emp_i...  | 0 row(s) affected                                          |
| 12 | 12:11:59 | CALL GetEmployeeById(102)                                                                  | 1 row(s) returned                                          |

### Procedure 3 – Update Salary

**Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.**

```

DROP PROCEDURE IF EXISTS IncreaseSalary;
DELIMITER $$

```

```

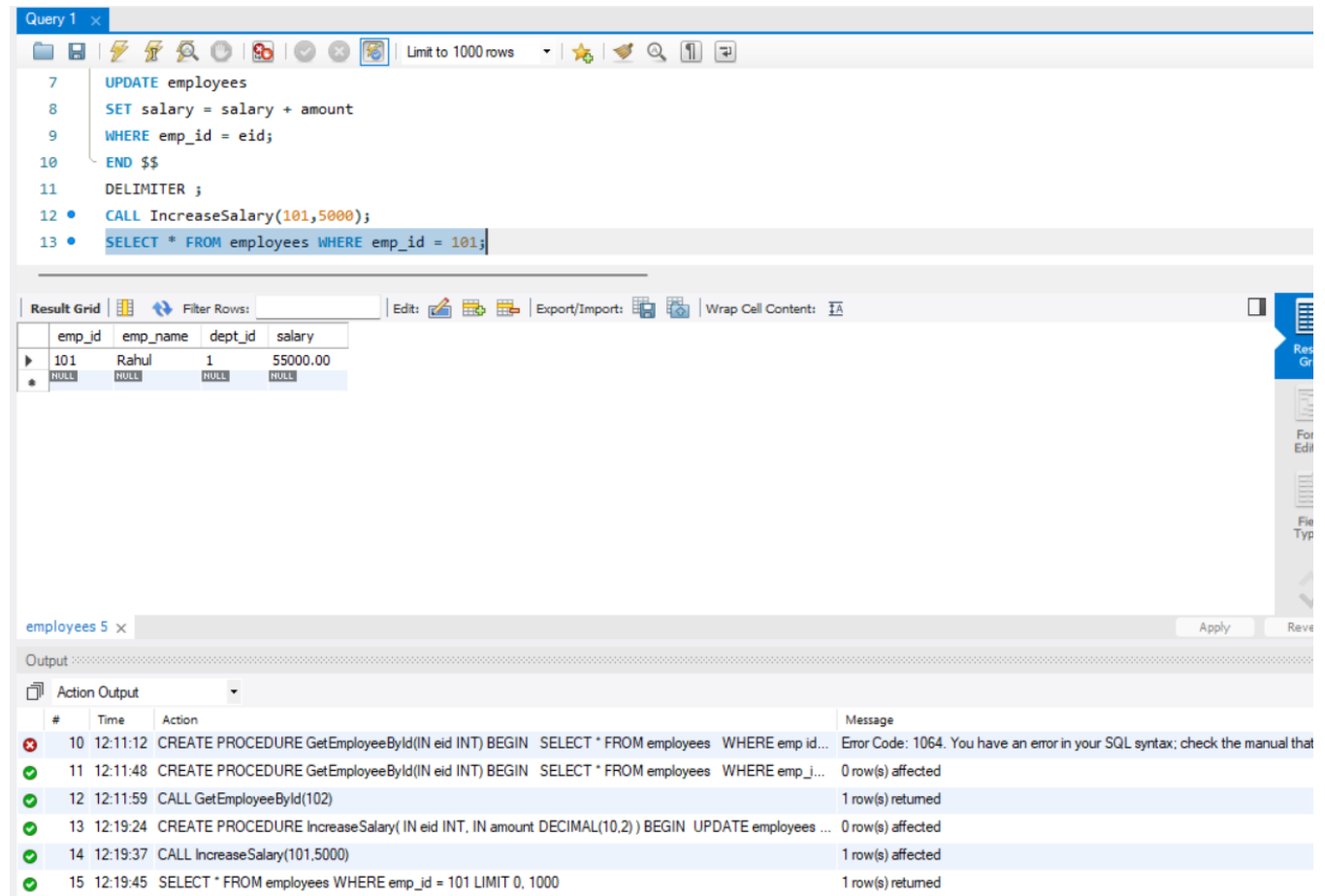
CREATE PROCEDURE IncreaseSalary(
    IN eid INT,
    IN amount DECIMAL(10,2)
)
BEGIN
    UPDATE employees
    SET salary = salary + amount
    WHERE emp_id = eid;
END $$

DELIMITER ;

```

```
CALL IncreaseSalary(101,5000);
SELECT * FROM employees WHERE emp_id=101;
```

## OUTPUT:



The screenshot shows a SQL IDE interface. The top window, titled 'Query 1', contains the following SQL code:

```
7 UPDATE employees
8 SET salary = salary + amount
9 WHERE emp_id = eid;
10 END $$
11 DELIMITER ;
12 • CALL IncreaseSalary(101,5000);
13 • SELECT * FROM employees WHERE emp_id = 101;
```

Below the query window is the 'Result Grid' showing the output of the query. It has columns for emp\_id, emp\_name, dept\_id, and salary. The first row shows emp\_id 101, emp\_name 'Rahul', dept\_id 1, and salary 55000.00. The second row shows NULL values for all columns.

| emp_id | emp_name | dept_id | salary   |
|--------|----------|---------|----------|
| 101    | Rahul    | 1       | 55000.00 |
| NULL   | NULL     | NULL    | NULL     |

At the bottom, the 'Output' window shows the execution log. It includes a table with columns for #, Time, Action, and Message. The log shows the execution of the SQL code, including the creation of procedures and the execution of the CALL and SELECT statements.

| #  | Time     | Action                                                                                            | Message                                                                       |
|----|----------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 10 | 12:11:12 | CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM employees WHERE emp id...        | Error Code: 1064. You have an error in your SQL syntax; check the manual that |
| 11 | 12:11:48 | CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM employees WHERE emp_j...         | 0 row(s) affected                                                             |
| 12 | 12:11:59 | CALL GetEmployeeById(102)                                                                         | 1 row(s) returned                                                             |
| 13 | 12:19:24 | CREATE PROCEDURE IncreaseSalary( IN eid INT, IN amount DECIMAL(10,2) ) BEGIN UPDATE employees ... | 0 row(s) affected                                                             |
| 14 | 12:19:37 | CALL IncreaseSalary(101,5000)                                                                     | 1 row(s) affected                                                             |
| 15 | 12:19:45 | SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000                                          | 1 row(s) returned                                                             |

## Procedure 4 – Aggregate Function

**Question: Create a procedure to calculate the average salary of all employees.**

```
DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$
```

```
CREATE PROCEDURE AvgSalary()
BEGIN
    SELECT AVG(salary) AS AverageSalary
    FROM employees;
END $$
```

```
DELIMITER ;
```

```
CALL AvgSalary();
```

## OUTPUT:

The screenshot displays a SQL IDE interface. The top pane shows a SQL script with the following code:

```
1 DELIMITER $$
2 CREATE PROCEDURE AvgSalary()
3 BEGIN
4     SELECT AVG(salary) AS AverageSalary
5     FROM employees;
6 END $$
7
```

The bottom pane shows the 'Result Grid' with one row of data:

| AverageSalary |
|---------------|
| 58600.000000  |

Below the result grid, the 'Output' tab is selected, showing a table of execution messages:

| #  | Time     | Action                                                                                            | Message           |
|----|----------|---------------------------------------------------------------------------------------------------|-------------------|
| 12 | 12:11:59 | CALL GetEmployeeById(102)                                                                         | 1 row(s) returned |
| 13 | 12:19:24 | CREATE PROCEDURE IncreaseSalary( IN eid INT, IN amount DECIMAL(10,2) ) BEGIN UPDATE employees ... | 0 row(s) affected |
| 14 | 12:19:37 | CALL IncreaseSalary(101,5000)                                                                     | 1 row(s) affected |
| 15 | 12:19:45 | SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000                                          | 1 row(s) returned |
| 16 | 12:24:22 | CREATE PROCEDURE AvgSalary() BEGIN SELECT AVG(salary) AS AverageSalary FROM employees; ...        | 0 row(s) affected |
| 17 | 12:24:29 | CALL AvgSalary()                                                                                  | 1 row(s) returned |

### Procedure 5 – JOIN

**Question: Create a procedure to display employee name, department name and salary. Run the department setup first.**

```
CREATE TABLE IF NOT EXISTS departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
INSERT IGNORE INTO departments VALUES  
(1,'CSE'),(2,'ECE'),(3,'EEE');
```

```
DROP PROCEDURE IF EXISTS EmployeeDepartment;  
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeDepartment()
```

```

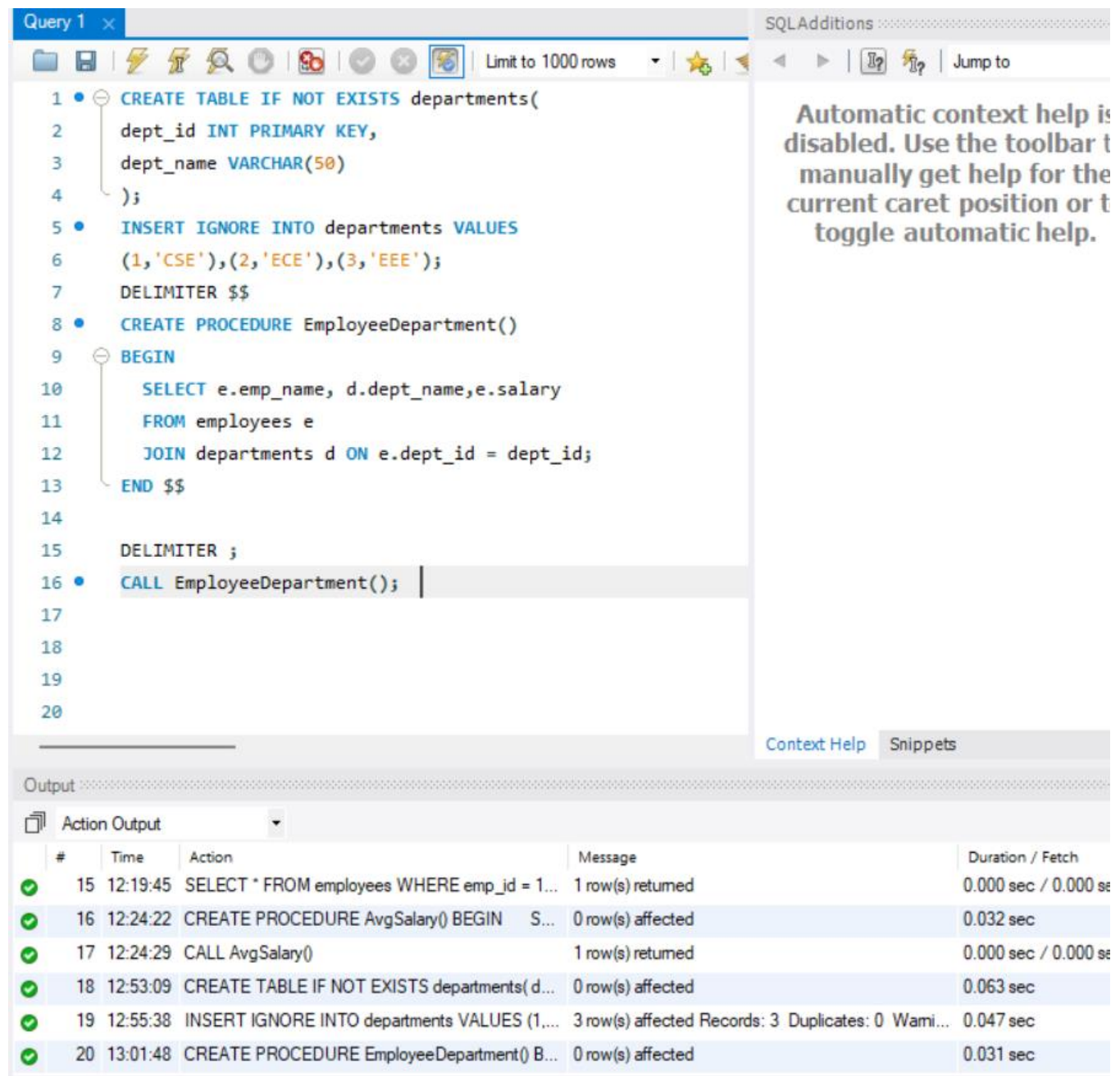
BEGIN
    SELECT e.emp_name, d.dept_name, e.salary
    FROM employees e
    JOIN departments d ON e.dept_id=d.dept_id;
END $$

```

DELIMITER ;

CALL EmployeeDepartment();

## OUTPUT:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and help. The main editor displays the following SQL code:

```

1 CREATE TABLE IF NOT EXISTS departments(
2     dept_id INT PRIMARY KEY,
3     dept_name VARCHAR(50)
4 );
5 INSERT IGNORE INTO departments VALUES
6     (1, 'CSE'), (2, 'ECE'), (3, 'EEE');
7 DELIMITER $$
8 CREATE PROCEDURE EmployeeDepartment()
9 BEGIN
10     SELECT e.emp_name, d.dept_name, e.salary
11     FROM employees e
12     JOIN departments d ON e.dept_id = dept_id;
13 END $$
14
15 DELIMITER ;
16 CALL EmployeeDepartment();
17
18
19
20

```

On the right side, there is a "SQLAdditions" panel with a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

Below the editor is the "Output" window, which shows a table of execution results:

| #  | Time     | Action                                       | Message                                            | Duration / Fetch      |
|----|----------|----------------------------------------------|----------------------------------------------------|-----------------------|
| 15 | 12:19:45 | SELECT * FROM employees WHERE emp_id = 1...  | 1 row(s) returned                                  | 0.000 sec / 0.000 sec |
| 16 | 12:24:22 | CREATE PROCEDURE AvgSalary() BEGIN S...      | 0 row(s) affected                                  | 0.032 sec             |
| 17 | 12:24:29 | CALL AvgSalary()                             | 1 row(s) returned                                  | 0.000 sec / 0.000 sec |
| 18 | 12:53:09 | CREATE TABLE IF NOT EXISTS departments( d... | 0 row(s) affected                                  | 0.063 sec             |
| 19 | 12:55:38 | INSERT IGNORE INTO departments VALUES (1,... | 3 row(s) affected Records: 3 Duplicates: 0 Wami... | 0.047 sec             |
| 20 | 13:01:48 | CREATE PROCEDURE EmployeeDepartment() B...   | 0 row(s) affected                                  | 0.031 sec             |

## Procedure 6 – GROUP BY and AVG

**Question: Create a procedure to display department-wise employee count and average salary.**

```
DROP PROCEDURE IF EXISTS DepartmentAverageSalary;  
DELIMITER $$
```

```
CREATE PROCEDURE DepartmentAverageSalary()  
BEGIN  
    SELECT d.dept_name,  
           COUNT(e.emp_id) AS EmployeeCount,  
           AVG(e.salary) AS AverageSalary  
    FROM employees e  
    JOIN departments d ON e.dept_id=d.dept_id  
    GROUP BY d.dept_id, d.dept_name;  
END $$
```

```
DELIMITER ;
```

```
CALL DepartmentAverageSalary();
```



## OUTPUT:

The screenshot displays a SQL IDE interface. The top pane shows a SQL query for Query 1:

```
7 FROM employees e
8 JOIN departments d ON e.dept_id = d.dept_id
9 GROUP BY d.dept_id, d.dept_name;
10 END $$
11 DELIMITER ;
12 CALL DepartmentAverageSalary();
13
```

The middle pane shows the Result Grid with the following data:

| dept_name | EmployeeCount | AverageSalary |
|-----------|---------------|---------------|
| CSE       | 2             | 55000.000000  |
| ECE       | 2             | 56500.000000  |
| EEE       | 1             | 70000.000000  |

The bottom pane shows the Action Output log with the following entries:

| #  | Time     | Action                                       | Message                                               | Duration / Fetch      |
|----|----------|----------------------------------------------|-------------------------------------------------------|-----------------------|
| 24 | 13:07:45 | INSERT IGNORE INTO departments VALUES (1,... | 0 row(s) affected, 3 warning(s): 1062 Duplicate en... | 0.000 sec             |
| 25 | 13:07:45 | DROP PROCEDURE IF EXISTS EmployeeDepart...   | 0 row(s) affected                                     | 0.016 sec             |
| 26 | 13:07:45 | CREATE PROCEDURE EmployeeDepartment() B...   | 0 row(s) affected                                     | 0.015 sec             |
| 27 | 13:07:45 | CALL EmployeeDepartment()                    | 5 row(s) returned                                     | 0.016 sec / 0.000 sec |
| 28 | 13:16:26 | CREATE PROCEDURE DepartmentAverageSalar...   | 0 row(s) affected                                     | 0.032 sec             |
| 29 | 13:16:26 | CALL DepartmentAverageSalary()               | 3 row(s) returned                                     | 0.000 sec / 0.000 sec |

### Procedure 7 – IF...ELSE

**Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.**

```
DROP PROCEDURE IF EXISTS SalaryStatus;
DELIMITER $$
```

```
CREATE PROCEDURE SalaryStatus(IN eid INT)
BEGIN
    DECLARE sal DECIMAL(10,2);

    SELECT salary INTO sal
```

```
FROM employees  
WHERE emp_id=eid;
```

```
IF sal >= 60000 THEN  
    SELECT 'High Salary' AS Status;  
ELSE  
    SELECT 'Low Salary' AS Status;  
END IF;  
END $$
```

```
DELIMITER ;
```

```
CALL SalaryStatus(102);
```

## OUTPUT:

The screenshot displays the SQL Developer interface. The main query editor shows a PL/SQL procedure named `SalaryStatus` with an input parameter `eid` of type `INT`. The procedure declares a variable `sal` of type `DECIMAL(10,2)`, selects the salary from the `employees` table, and filters by `emp_id = eid`. The toolbar includes icons for file operations, execution, and help. The right sidebar contains a message about automatic context help being disabled. The bottom output window shows a table of execution results.

**Query 1**

```
1 DELIMITER $$
2 CREATE PROCEDURE SalaryStatus(IN eid INT)
3 BEGIN
4     DECLARE sal DECIMAL(10,2);
5     SELECT salary INTO sal
6     FROM employees
7     WHERE emp_id = eid;
```

**Result Grid**

| Status      |
|-------------|
| High Salary |

**Output**

| #  | Time     | Action                                        | Message           | Duration / Fetch      |
|----|----------|-----------------------------------------------|-------------------|-----------------------|
| 26 | 13:07:45 | CREATE PROCEDURE EmployeeDepartment() B...    | 0 row(s) affected | 0.015 sec             |
| 27 | 13:07:45 | CALL EmployeeDepartment()                     | 5 row(s) returned | 0.016 sec / 0.000 sec |
| 28 | 13:16:26 | CREATE PROCEDURE DepartmentAverageSalar...    | 0 row(s) affected | 0.032 sec             |
| 29 | 13:16:26 | CALL DepartmentAverageSalary()                | 3 row(s) returned | 0.000 sec / 0.000 sec |
| 30 | 13:29:28 | CREATE PROCEDURE SalaryStatus(IN eid INT) ... | 0 row(s) affected | 0.000 sec             |
| 31 | 13:29:28 | CALL SalaryStatus(102)                        | 1 row(s) returned | 0.000 sec / 0.000 sec |

### Procedure 8 – OUT Parameter

**Question:** Return the total number of employees using an OUT parameter.

```
DROP PROCEDURE IF EXISTS EmployeeCount;
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeCount(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total
    FROM employees;
END $$
```

DELIMITER ;

```
CALL EmployeeCount(@total);  
SELECT @total AS TotalEmployees;
```

**OUTPUT:**

The screenshot displays the SQL Server Enterprise Manager interface. The 'Query 1' window shows the following SQL code:

```
1 DELIMITER $$  
2 CREATE PROCEDURE EmployeeCount(OUT total INT)  
3 BEGIN  
4     SELECT COUNT(*) INTO total  
5     FROM employees;  
6 END $$  
7 DELIMITER ;
```

The 'Result Grid' shows the output of the query:

| TotalEmployees |
|----------------|
| 5              |

The 'Output' window shows the execution results of the query, including the duration and fetch time for each row.

| #  | Time     | Action                                        | Message           | Duration / Fetch      |
|----|----------|-----------------------------------------------|-------------------|-----------------------|
| 29 | 13:16:26 | CALL DepartmentAverageSalary()                | 3 row(s) returned | 0.000 sec / 0.000 sec |
| 30 | 13:29:28 | CREATE PROCEDURE SalaryStatus(IN eid INT) ... | 0 row(s) affected | 0.000 sec             |
| 31 | 13:29:28 | CALL SalaryStatus(102)                        | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 32 | 13:35:25 | CREATE PROCEDURE EmployeeCount(OUT tot...     | 0 row(s) affected | 0.016 sec             |
| 33 | 13:35:25 | CALL EmployeeCount(@total)                    | 1 row(s) affected | 0.015 sec             |
| 34 | 13:35:25 | SELECT @total AS TotalEmployees LIMIT 0, 1000 | 1 row(s) returned | 0.000 sec / 0.000 sec |

### Procedure 9 – INOUT Parameter

**Question: Receive a bonus through an INOUT parameter and add 5000 to it.**

```
DROP PROCEDURE IF EXISTS AddBonus;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
```

```
BEGIN
```

```
    SET bonus = bonus + 5000;
```

```
END $$
```

```
DELIMITER ;
```

```
SET @bonus=10000;
```

```
CALL AddBonus(@bonus);
```

```
SELECT @bonus AS UpdatedBonus;
```

**OUTPUT:**

Query 1

Limit to 1000 rows

```

1 DELIMITER $$
2 CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
3 BEGIN
4     SET bonus = bonus + 5000;
5 END $$
6 DELIMITER ;
7 SET @bonus = 10000;

```

Result Grid

| UpdatedBonus |
|--------------|
| 15000.00     |

Filter Rows:

Export:

Wrap Cell

Result Grid

Form Editor

Field Types

Result 11

Read Only

Context Help

Snippets

Output

Action Output

| #  | Time     | Action                                        | Message           | Duration / Fetch      |
|----|----------|-----------------------------------------------|-------------------|-----------------------|
| 33 | 13:35:25 | CALL EmployeeCount(@total)                    | 1 row(s) affected | 0.015 sec             |
| 34 | 13:35:25 | SELECT @total AS TotalEmployees LIMIT 0, 1000 | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 35 | 13:44:37 | CREATE PROCEDURE AddBonus(INOUT bonus...      | 0 row(s) affected | 0.015 sec             |
| 36 | 13:44:37 | SET @bonus = 10000                            | 0 row(s) affected | 0.000 sec             |
| 37 | 13:44:37 | CALL AddBonus(@bonus)                         | 0 row(s) affected | 0.000 sec             |
| 38 | 13:44:37 | SELECT @bonus AS UpdatedBonus LIMIT 0, 1000   | 1 row(s) returned | 0.016 sec / 0.000 sec |

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

### Procedure 10 – Transaction

**Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.**

```
DROP PROCEDURE IF EXISTS TransferSalaryAmount;  
DELIMITER $$
```

```
CREATE PROCEDURE TransferSalaryAmount(  
    IN fromEmp INT,  
    IN toEmp INT,  
    IN amount DECIMAL(10,2)  
)  
BEGIN
```

```
    START TRANSACTION;
```

```
    UPDATE employees  
    SET salary=salary-amount  
    WHERE emp_id=fromEmp;
```

```
    UPDATE employees  
    SET salary=salary+amount  
    WHERE emp_id=toEmp;
```

```
    COMMIT;  
END $$
```

```
DELIMITER ;
```

```
CALL TransferSalaryAmount(101,102,2000);  
SELECT * FROM employees WHERE emp_id IN (101,102);
```



## OUTPUT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and help, along with a 'Limit to 1000 rows' dropdown. The query editor shows a SQL script with a CREATE PROCEDURE statement. Below the editor is a 'Result Grid' showing a table with columns emp\_id, emp\_name, dept\_id, and salary. The output window at the bottom shows a log of SQL actions and their results.

**Query 1**

```
1 DELIMITER $$
2
3 CREATE PROCEDURE TransferSalaryAmount(
4     IN fromEmp INT,
5     IN toEmp INT,
6     IN amount DECIMAL(10,2)
7 )
```

**Result Grid**

| emp_id | emp_name | dept_id | salary   |
|--------|----------|---------|----------|
| 101    | Rahul    | 1       | 53000.00 |
| 102    | Priya    | 2       | 67000.00 |
| NULL   | NULL     | NULL    | NULL     |

**SQLAdditions**

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

**employees 12**

Apply Context Help Snippets

**Output**

Action Output

| #    | Time     | Action                                       | Message           | Duration / Fetch      |
|------|----------|----------------------------------------------|-------------------|-----------------------|
| ✓ 36 | 13:44:37 | SET @bonus = 10000                           | 0 row(s) affected | 0.000 sec             |
| ✓ 37 | 13:44:37 | CALL AddBonus(@bonus)                        | 0 row(s) affected | 0.000 sec             |
| ✓ 38 | 13:44:37 | SELECT @bonus AS UpdatedBonus LIMIT 0, 1000  | 1 row(s) returned | 0.016 sec / 0.000 sec |
| ✓ 39 | 13:50:14 | CREATE PROCEDURE TransferSalaryAmount( ...   | 0 row(s) affected | 0.046 sec             |
| ✓ 40 | 13:50:14 | CALL TransferSalaryAmount(101,102,2000)      | 0 row(s) affected | 0.016 sec             |
| ✓ 41 | 13:50:14 | SELECT * FROM employees WHERE emp_id IN (... | 2 row(s) returned | 0.000 sec / 0.000 sec |



#### 4. Useful Procedure Commands

SHOW PROCEDURE STATUS

WHERE Db='employeeedb1';

SHOW CREATE PROCEDURE GetEmployees;

DROP PROCEDURE IF EXISTS GetEmployees;

#### 5. Why DELIMITER is Used

DELIMITER temporarily changes the statement terminator in MySQL Workbench. It prevents the semicolons inside BEGIN...END from being interpreted as the end of the entire CREATE PROCEDURE statement.

DELIMITER \$\$

CREATE PROCEDURE Example()

BEGIN

    SELECT \* FROM employees;

END \$\$

DELIMITER ;

**OUTPUT:**

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DELIMITER $$
2 CREATE PROCEDURE Example()
3 BEGIN
4     SELECT * FROM employees;
5 END $$
6 DELIMITER ;
7 CALL EXAMPLE();

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid Filter Rows: Export: Wrap Cell Content

|   | emp_id | emp_name | dept_id | salary   |
|---|--------|----------|---------|----------|
| ▶ | 101    | Rahul    | 1       | 53000.00 |
|   | 102    | Priya    | 2       | 67000.00 |
|   | 103    | Anil     | 1       | 55000.00 |
|   | 104    | Sneha    | 3       | 70000.00 |
|   | 105    | Kiran    | 2       | 48000.00 |

Result Grid Form Editor Field Types

Result 13 x Read Only Context Help Snippets

Output

Action Output

| #    | Time     | Action                                       | Message           | Duration / Fetch      |
|------|----------|----------------------------------------------|-------------------|-----------------------|
| ✓ 38 | 13:44:37 | SELECT @bonus AS UpdatedBonus LIMIT 0, 1000  | 1 row(s) returned | 0.016 sec / 0.000 sec |
| ✓ 39 | 13:50:14 | CREATE PROCEDURE TransferSalaryAmount( ...   | 0 row(s) affected | 0.046 sec             |
| ✓ 40 | 13:50:14 | CALL TransferSalaryAmount(101,102,2000)      | 0 row(s) affected | 0.016 sec             |
| ✓ 41 | 13:50:14 | SELECT * FROM employees WHERE emp_id IN (... | 2 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 42 | 14:02:44 | CREATE PROCEDURE Example() BEGIN SEL...      | 0 row(s) affected | 0.032 sec             |
| ✓ 43 | 14:04:41 | CALL EXAMPLE()                               | 5 row(s) returned | 0.000 sec / 0.000 sec |

## 6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeesBySalary(IN min_salary DECIMAL(10,2))
```

```
BEGIN
```

```
    SELECT emp_id, emp_name, salary, dept_id
```

```
    FROM employees
```

```
WHERE salary > min_salary;
```

```
END //
```

```
DELIMITER ;
```

```
CALL GetEmployeesBySalary(60000);
```

## OUTPUT:

The screenshot shows the SQL Server Enterprise Manager interface. The 'Query 1' window displays the following SQL code:

```
1 DELIMITER $$
2 CREATE PROCEDURE GetEmployeesBySalary(IN min_salary Decima
3 BEGIN
4     SELECT emp_id, emp_name, salary, dept_id
5     FROM employees
6     WHERE salary > min_salary;
7 END $$
```

The 'Result Grid' shows the output of the query:

| emp_id | emp_name | salary   | dept_id |
|--------|----------|----------|---------|
| 102    | Priya    | 67000.00 | 2       |
| 104    | Sneha    | 70000.00 | 3       |

The 'Output' window shows the execution log:

| # | Time     | Action                                       | Message           | Duration / Fetch      |
|---|----------|----------------------------------------------|-------------------|-----------------------|
| 1 | 20:25:21 | SHOW DATABASES                               | 9 row(s) returned | 0.016 sec / 0.000 sec |
| 2 | 20:56:36 | USE employeeedb1                             | 0 row(s) affected | 0.000 sec             |
| 3 | 21:02:29 | CREATE PROCEDURE GetEmployeesBySalary(IN ... | 0 row(s) affected | 0.046 sec             |
| 4 | 21:03:36 | CALL GetEmployeesBySalary(60000)             | 2 row(s) returned | 0.015 sec / 0.000 sec |

2. Create a procedure to update an employee's department using IN parameters.

DELIMITER //

CREATE PROCEDURE UpdateEmployeeDepartment(

IN p\_emp\_id INT,

IN p\_dept\_id INT

)

BEGIN

UPDATE employees

```
SET dept_id = p_dept_id  
WHERE emp_id = p_emp_id;  
END //
```

```
DELIMITER ;  
CALL UpdateEmployeeDepartment(101, 3);  
SELECT * FROM employees  
WHERE emp_id = 101;
```

## OUTPUT:

The screenshot displays the SQL Developer interface. The top toolbar includes icons for file operations, execution, and help. The 'Query 1' window shows the following SQL code:

```
9      WHERE emp_id = p_emp_id;
10  END $$
11  DELIMITER ;
12  • CALL UpdateEmployeeDepartment(101, 3);
13
14  • SELECT * FROM employees
15  WHERE emp_id = 101;
```

To the right of the query window, a text box states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

Below the query window is the 'Result Grid' showing the results of the last query:

| emp_id | emp_name | dept_id | salary   |
|--------|----------|---------|----------|
| 101    | Rahul    | 3       | 53000.00 |
| NULL   | NULL     | NULL    | NULL     |

At the bottom, the 'Output' window shows the 'Action Output' log:

| #   | Time     | Action                                      | Message           | Duration / Fetch      |
|-----|----------|---------------------------------------------|-------------------|-----------------------|
| ✓ 2 | 20:56:36 | USE employeeedb1                            | 0 row(s) affected | 0.000 sec             |
| ✓ 3 | 21:02:29 | CREATE PROCEDURE GetEmployeesBySalary(l...  | 0 row(s) affected | 0.046 sec             |
| ✓ 4 | 21:03:36 | CALL GetEmployeesBySalary(60000)            | 2 row(s) returned | 0.015 sec / 0.000 sec |
| ✓ 5 | 21:14:49 | CREATE PROCEDURE UpdateEmployeeDepart...    | 0 row(s) affected | 0.031 sec             |
| ✓ 6 | 21:16:14 | CALL UpdateEmployeeDepartment(101, 3)       | 1 row(s) affected | 0.031 sec             |
| ✓ 7 | 21:17:42 | SELECT * FROM employees WHERE emp_id = 1... | 1 row(s) returned | 0.000 sec / 0.000 sec |

3. Create a procedure to return the maximum salary using an OUT parameter.

```
DELIMITER //
```

```
CREATE PROCEDURE GetMaximumSalary(OUT max_salary DECIMAL(10,2))
```

```
BEGIN
```

```
    SELECT MAX(salary)
```

```
    INTO max_salary
```

```
    FROM employees;
```

END //

DELIMITER ;

CALL GetMaximumSalary(@max\_salary);

SELECT @max\_salary AS Maximum\_Salary;

## OUTPUT:

The screenshot shows the SQL Developer interface. The top toolbar includes icons for file operations, execution, and help. The main query window displays the following SQL code:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE GetMaximumSalary(OUT max_salary DECIMAL(10,2))
4 BEGIN
5     SELECT MAX(salary)
6     INTO max_salary
7     FROM employees;
```

Below the query window, the 'Result Grid' tab is active, showing a single row of results:

| Maximum_Salary |
|----------------|
| 70000.00       |

On the right side, a 'SQLAdditions' panel displays a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

At the bottom, the 'Output' window shows the 'Action Output' tab with a table of execution results:

| #    | Time     | Action                                      | Message           | Duration / Fetch      |
|------|----------|---------------------------------------------|-------------------|-----------------------|
| ✓ 5  | 21:14:49 | CREATE PROCEDURE UpdateEmployeeDepart...    | 0 row(s) affected | 0.031 sec             |
| ✓ 6  | 21:16:14 | CALL UpdateEmployeeDepartment(101, 3)       | 1 row(s) affected | 0.031 sec             |
| ✓ 7  | 21:17:42 | SELECT * FROM employees WHERE emp_id = 1... | 1 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 8  | 21:22:49 | CREATE PROCEDURE GetMaximumSalary(OUT ...   | 0 row(s) affected | 0.031 sec             |
| ✓ 9  | 21:23:04 | CALL GetMaximumSalary(@max_salary)          | 1 row(s) affected | 0.000 sec             |
| ✓ 10 | 21:23:04 | SELECT @max_salary AS Maximum_Salary LIM... | 1 row(s) returned | 0.000 sec / 0.000 sec |

4. Create a procedure to return the employee count of a specified department.

**DELIMITER //**

```
CREATE PROCEDURE GetEmployeeCount(  
    IN p_dept_id INT,  
    OUT emp_count INT  
)
```



BEGIN

SELECT COUNT(\*)

INTO emp\_count

FROM employees

WHERE dept\_id = p\_dept\_id;

END //

DELIMITER ;

CALL GetEmployeeCount(2, @count);

SELECT @count AS Employee\_Count;

## OUTPUT:

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window titled 'Query 1' containing the following SQL code:

```
2
3 CREATE PROCEDURE GetEmployeeCount(
4     IN p_dept_id INT,
5     OUT emp_count INT
6 )
7 BEGIN
8     SELECT COUNT(*)
```

The bottom pane shows the 'Result Grid' with a single row of data:

| Employee_Count |
|----------------|
| 2              |

Below the result grid, the 'Output' pane shows the 'Action Output' table, which lists the execution steps and their results:

| #  | Time     | Action                                        | Message           | Duration / Fetch      |
|----|----------|-----------------------------------------------|-------------------|-----------------------|
| 8  | 21:22:49 | CREATE PROCEDURE GetMaximumSalary(OUT ...     | 0 row(s) affected | 0.031 sec             |
| 9  | 21:23:04 | CALL GetMaximumSalary(@max_salary)            | 1 row(s) affected | 0.000 sec             |
| 10 | 21:23:04 | SELECT @max_salary AS Maximum_Salary LIM...   | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 11 | 21:26:06 | CREATE PROCEDURE GetEmployeeCount( I...       | 0 row(s) affected | 0.031 sec             |
| 12 | 21:26:39 | CALL GetEmployeeCount(2, @count)              | 1 row(s) affected | 0.000 sec             |
| 13 | 21:26:58 | SELECT @count AS Employee_Count LIMIT 0, 1... | 1 row(s) returned | 0.000 sec / 0.000 sec |

5. Create a procedure using JOIN to display employee and department details.

DELIMITER //

```
CREATE PROCEDURE GetEmployeeDepartmentDetails()
```

```
BEGIN
```

```
    SELECT
```

```
e.emp_id,  
e.emp_name,  
e.salary,  
d.dept_id,  
d.dept_name
```

```
FROM employees e
```

```
INNER JOIN departments d
```

```
ON e.dept_id = d.dept_id;
```

```
END //
```

```
DELIMITER ;
```

```
CALL GetEmployeeDepartmentDetails();
```

## OUTPUT:

The screenshot shows the SQL Developer interface. The 'Query 1' window contains the following SQL code:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE GetEmployeeDepartmentDetails()
4 BEGIN
5     SELECT
6         e.emp_id,
7         e.emp_name,
```

The 'Result Grid' shows the following data:

| emp_id | emp_name | salary   | dept_id | dept_name |
|--------|----------|----------|---------|-----------|
| 101    | Rahul    | 53000.00 | 3       | EEE       |
| 102    | Priya    | 67000.00 | 2       | ECE       |
| 103    | Anil     | 55000.00 | 1       | CSE       |
| 104    | Sneha    | 70000.00 | 3       | EEE       |
| 105    | Kiran    | 48000.00 | 2       | ECE       |

The 'Output' window shows the following execution log:

| #  | Time     | Action                                        | Message           | Duration / Fetch      |
|----|----------|-----------------------------------------------|-------------------|-----------------------|
| 10 | 21:23:04 | SELECT @max_salary AS Maximum_Salary LIM...   | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 11 | 21:26:06 | CREATE PROCEDURE GetEmployeeCount( I...       | 0 row(s) affected | 0.031 sec             |
| 12 | 21:26:39 | CALL GetEmployeeCount(2, @count)              | 1 row(s) affected | 0.000 sec             |
| 13 | 21:26:58 | SELECT @count AS Employee_Count LIMIT 0, 1... | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 14 | 21:31:28 | CREATE PROCEDURE GetEmployeeDepartment...     | 0 row(s) affected | 0.047 sec             |
| 15 | 21:31:41 | CALL GetEmployeeDepartmentDetails()           | 5 row(s) returned | 0.000 sec / 0.000 sec |

6. Create a procedure to display departments whose average salary is greater than 55000.

DELIMITER //

CREATE PROCEDURE GetHighAverageSalaryDepartments()

BEGIN

```
SELECT
    d.dept_id,
    d.dept_name,
    AVG(e.salary) AS average_salary
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 55000;

END //

DELIMITER ;

CALL GetHighAverageSalaryDepartments();
```

## OUTPUT:

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window titled 'Query 1' containing the following SQL code:

```
3 • CREATE PROCEDURE GetHighAverageSalaryDepartments()  
4 BEGIN  
5     SELECT  
6         d.dept_id,  
7         d.dept_name,  
8         AVG(e.salary) AS average_salary  
9     FROM employees e
```

The bottom pane shows the 'Result Grid' with the following data:

| dept_id | dept_name | average_salary |
|---------|-----------|----------------|
| 3       | EEE       | 61500.000000   |
| 2       | ECE       | 57500.000000   |

On the right side of the interface, a message states: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

At the bottom, the 'Output' pane shows a table of execution results:

| #    | Time     | Action                                        | Message           | Duration / Fetch      |
|------|----------|-----------------------------------------------|-------------------|-----------------------|
| ✓ 12 | 21:26:39 | CALL GetEmployeeCount(2, @count)              | 1 row(s) affected | 0.000 sec             |
| ✓ 13 | 21:26:58 | SELECT @count AS Employee_Count LIMIT 0, 1... | 1 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 14 | 21:31:28 | CREATE PROCEDURE GetEmployeeDepartment...     | 0 row(s) affected | 0.047 sec             |
| ✓ 15 | 21:31:41 | CALL GetEmployeeDepartmentDetails()           | 5 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 16 | 21:36:17 | CREATE PROCEDURE GetHighAverageSalaryD...     | 0 row(s) affected | 0.031 sec             |
| ✓ 17 | 21:36:31 | CALL GetHighAverageSalaryDepartments()        | 2 row(s) returned | 0.016 sec / 0.000 sec |

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

DELIMITER //

CREATE PROCEDURE ClassifySalary(  
 IN p\_emp\_id INT

```
)  
  
BEGIN  
  
    DECLARE emp_salary DECIMAL(10,2);  
  
    SELECT salary  
    INTO emp_salary  
    FROM employees  
    WHERE emp_id = p_emp_id;  
  
    IF emp_salary > 70000 THEN  
        SELECT emp_salary AS Salary, 'High' AS Salary_Category;  
    ELSEIF emp_salary >= 40000 THEN  
        SELECT emp_salary AS Salary, 'Medium' AS Salary_Category;  
    ELSE  
        SELECT emp_salary AS Salary, 'Low' AS Salary_Category;  
    END IF;  
  
END //
```

```
DELIMITER ;  
  
CALL ClassifySalary(101);
```

## OUTPUT:

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and help. The query editor displays the following SQL code:

```
1 DELIMITER //  
2  
3 CREATE PROCEDURE ClassifySalary(  
4     IN p_emp_id INT  
5 )  
6 BEGIN  
7     DECLARE emp_salary DECIMAL(10,2);
```

Below the query editor is a 'Result Grid' showing a single row of data:

| Salary   | Salary_Category |
|----------|-----------------|
| 53000.00 | Medium          |

To the right of the query editor is a 'SQLAdditions' panel with a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

At the bottom, the 'Output' window shows a table of execution results:

| #    | Time     | Action                                     | Message           | Duration / Fetch      |
|------|----------|--------------------------------------------|-------------------|-----------------------|
| ✓ 14 | 21:31:28 | CREATE PROCEDURE GetEmployeeDepartment...  | 0 row(s) affected | 0.047 sec             |
| ✓ 15 | 21:31:41 | CALL GetEmployeeDepartmentDetails()        | 5 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 16 | 21:36:17 | CREATE PROCEDURE GetHighAverageSalaryD...  | 0 row(s) affected | 0.031 sec             |
| ✓ 17 | 21:36:31 | CALL GetHighAverageSalaryDepartments()     | 2 row(s) returned | 0.016 sec / 0.000 sec |
| ✓ 18 | 21:39:12 | CREATE PROCEDURE ClassifySalary( IN p_e... | 0 row(s) affected | 0.031 sec             |
| ✓ 19 | 21:39:27 | CALL ClassifySalary(101)                   | 1 row(s) returned | 0.000 sec / 0.000 sec |

8. Create a procedure that accepts a department ID and returns its average salary.

DELIMITER //

CREATE PROCEDURE GetAverageSalary(  
 IN p\_dept\_id INT,  
 OUT avg\_salary DECIMAL(10,2)  
)



BEGIN

SELECT AVG(salary)

INTO avg\_salary

FROM employees

WHERE dept\_id = p\_dept\_id;

END //

DELIMITER ;

CALL GetAverageSalary(2, @average);

SELECT @average AS Average\_Salary;

## OUTPUT:

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and help. The query editor displays the following SQL code:

```
2  
3 CREATE PROCEDURE GetAverageSalary(  
4     IN p_dept_id INT,  
5     OUT avg_salary DECIMAL(10,2)  
6 )  
7 BEGIN  
8     SELECT AVG(salary)
```

Below the query editor is a result grid showing one row of data:

| Average_Salary |
|----------------|
| 57500.00       |

On the right side, a context help panel displays the text: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

At the bottom, the "Output" window shows a table of execution results:

| #    | Time     | Action                                         | Message           | Duration / Fetch      |
|------|----------|------------------------------------------------|-------------------|-----------------------|
| ✓ 17 | 21:36:31 | CALL GetHighAverageSalaryDepartments()         | 2 row(s) returned | 0.016 sec / 0.000 sec |
| ✓ 18 | 21:39:12 | CREATE PROCEDURE ClassifySalary( IN p_e...     | 0 row(s) affected | 0.031 sec             |
| ✓ 19 | 21:39:27 | CALL ClassifySalary(101)                       | 1 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 20 | 21:55:20 | CREATE PROCEDURE GetAverageSalary( IN ...      | 0 row(s) affected | 0.047 sec             |
| ✓ 21 | 21:55:34 | CALL GetAverageSalary(2, @average)             | 1 row(s) affected | 0.015 sec             |
| ✓ 22 | 21:55:57 | SELECT @average AS Average_Salary LIMIT 0, ... | 1 row(s) returned | 0.000 sec / 0.000 sec |

9. Create a procedure using INOUT to apply a bonus to an input salary.

DELIMITER //

```
CREATE PROCEDURE ApplyBonus(  
    INOUT p_salary DECIMAL(10,2)  
)
```

```
)
```

```
BEGIN
```

```
SET p_salary = p_salary + (p_salary * 0.10);  
END //
```

```
DELIMITER ;
```

```
SET @salary = 50000;
```

```
CALL ApplyBonus(@salary);
```

```
SELECT @salary AS Salary_After_Bonus;
```

## OUTPUT:

The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and help. The query editor shows the following SQL code:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE ApplyBonus(
4     INOUT p_salary DECIMAL(10,2)
5 )
6 BEGIN
7     SET p_salary = p_salary + (p_salary * 0.10);
```

Below the query editor, the 'Result Grid' tab is active, showing a single row of data:

| Salary_After_Bonus |
|--------------------|
| 55000.00           |

To the right of the query editor, a text box states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

The bottom section of the IDE shows the 'Output' window with the 'Action Output' tab selected. It contains a table of execution logs:

| #    | Time     | Action                                          | Message           | Duration / Fetch      |
|------|----------|-------------------------------------------------|-------------------|-----------------------|
| ✓ 21 | 21:55:34 | CALL GetAverageSalary(2, @average)              | 1 row(s) affected | 0.015 sec             |
| ✓ 22 | 21:55:57 | SELECT @average AS Average_Salary LIMIT 0, ...  | 1 row(s) returned | 0.000 sec / 0.000 sec |
| ✓ 23 | 21:59:22 | CREATE PROCEDURE ApplyBonus( INOUT p...         | 0 row(s) affected | 0.015 sec             |
| ✓ 24 | 21:59:59 | SET @salary = 50000                             | 0 row(s) affected | 0.016 sec             |
| ✓ 25 | 21:59:59 | CALL ApplyBonus(@salary)                        | 0 row(s) affected | 0.015 sec             |
| ✓ 26 | 21:59:59 | SELECT @salary AS Salary_After_Bonus LIMIT 0... | 1 row(s) returned | 0.000 sec / 0.000 sec |

10. Create a procedure that performs two related updates inside a transaction.

DELIMITER //

CREATE PROCEDURE UpdateEmployeeDetails(

IN p\_emp\_id INT,

IN p\_dept\_id INT

```
)  
  
BEGIN  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  
    BEGIN  
  
        ROLLBACK;  
  
    END;  
  
  
    START TRANSACTION;  
  
  
    -- Update salary by 10%  
  
    UPDATE employees  
  
    SET salary = salary * 1.10  
  
    WHERE emp_id = p_emp_id;  
  
  
    -- Update department  
  
    UPDATE employees  
  
    SET dept_id = p_dept_id  
  
    WHERE emp_id = p_emp_id;  
  
  
    COMMIT;  
  
END //  
  
  
DELIMITER ;  
  
CALL UpdateEmployeeDetails(101, 3);  
  
SELECT emp_id, emp_name, salary, dept_id  
  
FROM employees  
  
WHERE emp_id = 101;
```

## OUTPUT:

The screenshot displays the SQL Developer interface. The top toolbar includes icons for file operations, execution, and help. The main query window, titled 'Query 1', contains the following SQL code:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE UpdateEmployeeDetails(
4     IN p_emp_id INT,
5     IN p_dept_id INT
6 )
7 BEGIN
```

Below the query window is the 'Result Grid' tab, which shows a table with the following data:

| emp_id | emp_name | salary   | dept_id |
|--------|----------|----------|---------|
| 101    | Rahul    | 58300.00 | 3       |
| *      | NULL     | NULL     | NULL    |

To the right of the query window is a 'SQLAdditions' panel with a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

At the bottom of the interface is the 'Output' window, which shows the 'Action Output' tab. It contains a table with the following data:

| #  | Time     | Action                                          | Message           | Duration / Fetch      |
|----|----------|-------------------------------------------------|-------------------|-----------------------|
| 24 | 21:59:59 | SET @salary = 50000                             | 0 row(s) affected | 0.016 sec             |
| 25 | 21:59:59 | CALL ApplyBonus(@salary)                        | 0 row(s) affected | 0.015 sec             |
| 26 | 21:59:59 | SELECT @salary AS Salary_After_Bonus LIMIT 0... | 1 row(s) returned | 0.000 sec / 0.000 sec |
| 27 | 22:04:07 | CREATE PROCEDURE UpdateEmployeeDetails( ...     | 0 row(s) affected | 0.047 sec             |
| 28 | 22:04:25 | CALL UpdateEmployeeDetails(101, 3)              | 0 row(s) affected | 0.032 sec             |
| 29 | 22:06:09 | SELECT emp_id, emp_name, salary, dept_id FRO... | 1 row(s) returned | 0.000 sec / 0.000 sec |

## 7. Common Errors

**Procedure already exists:** Use DROP PROCEDURE IF EXISTS before recreating it.

**Syntax error near BEGIN:** Check DELIMITER \$\$ and DELIMITER ;.

**Table does not exist:** Run USE employeeedb1; and SHOW TABLES;.

**Unknown column:** Check names using DESCRIBE employees;